Classes in JavaScript

A class is a template for creating objects. It bundles data (properties) and behavior (methods) together. Classes were introduced in ES6 and are syntactic sugar over JavaScript's prototype-based inheritance.

Basic syntax

```
class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }

  greet() {
    return `Hi, I'm ${this.name} and I'm ${this.age}.`;
  }
}

const p = new Person("Rahim", 25);
console.log(p.greet()); // Hi, I'm Rahim and I'm 25.
```

- `constructor` runs automatically when you use `new`.
- Methods defined in the class are shared on the prototype (not copied per instance).
- You must use `new`; calling `Person()` directly throws an error.

Fields (properties)

```
class Counter {
  count = 0;            // public field
  #secret = 42;         // private field (only accessible inside the class)

  increment() {
    this.count++;
    return this;
  }

  getSecret() {
    return this.#secret;
  }
}

const c = new Counter();
c.increment().increment();
console.log(c.count);     // 2
console.log(c.getSecret()); // 42
// console.log(c.#secret); // SyntaxError
```

Getters and setters

```
class Circle {
  constructor(radius) {
    this.radius = radius;
  }

  get area() {
    return Math.PI * this.radius ** 2;
  }

  set diameter(d) {
    this.radius = d / 2;
  }
}

const c = new Circle(5);
console.log(c.area);   // 78.53...
c.diameter = 20;
console.log(c.radius); // 10
```

Static members

Belong to the class itself, not to instances.

```
class MathUtil {
  static PI = 3.14159;

  static square(x) {
    return x * x;
  }
}

console.log(MathUtil.square(4)); // 16
console.log(MathUtil.PI);        // 3.14159
```

You can also have `static #private` members and `static { ... }` initialization blocks.

Inheritance

```
class Animal {
  constructor(name) {
    this.name = name;
  }

  speak() {
    return `${this.name} makes a sound.`;
  }
}
```

```
class Dog extends Animal {
  constructor(name, breed) {
    super(name);          // must call before using `this`
    this.breed = breed;
  }

  speak() {
    return `${super.speak()} Woof!`;
  }
}

const d = new Dog("Rex", "Labrador");
console.log(d.speak()); // Rex makes a sound. Woof!
console.log(d instanceof Animal); // true
```

Private methods

```
class Account {
  #balance = 0;

  deposit(amount) {
    this.#validate(amount);
    this.#balance += amount;
  }

  #validate(amount) {
    if (amount <= 0) throw new Error("Invalid amount");
  }

  get balance() {
    return this.#balance;
  }
}
```

The `this` gotcha

Methods lose `this` when passed as callbacks:

```
class Button {
  label = "Click";

  handleClick() {
    console.log(this.label);
  }

  // Fix: arrow function field keeps `this` bound
  handleClickBound = () => {
    console.log(this.label);
  };
}
```

```
const b = new Button();
const fn = b.handleClick;
// fn();               // TypeError: this is undefined
const fn2 = b.handleClickBound;
fn2();                 // "Click"
```

Alternative: `this.handleClick = this.handleClick.bind(this)` in the constructor.

Quick facts

Feature	Syntax
Constructor	`constructor() {}`
Public field	`name = "x"`
Private field/method	`#name`
Static	`static name`
Getter/setter	`get x() {}` / `set x(v) {}`
Inherit	`class B extends A`
Parent call	`super(...)` / `super.method()`
Type check	`obj instanceof Class`

Other things worth knowing

- Classes are **not hoisted** like functions: you can't use one before its declaration.
- Class bodies run in **strict mode** automatically.
- Class expressions work too: `const Foo = class { ... }`.
- `typeof MyClass` is `"function"`, since a class is a special function.

If you'd like, I can go deeper into any part, such as mixins, `Symbol.iterator` in classes, or how classes map to prototypes under the hood.